

From Spreadsheets to Structured Data

Module 3 — Relational Databases & SQL for Analytics

Get to know Lana — Fun Fact



I Love to Cook!

- I am largely self-taught but rooted in family tradition
- Often cook for the family, but also just really love baking as a hobby, especially bread, scones, biscuits, and homemade pasta from scratch
- I find a lot of inspiration in flavors from my travels around the world

The "Contract"

Module 1: Your environment is a contract



+ requirements.txt



= "it works on any machine"

Module 2: Your pipeline functions are contracts



= "you know where it broke"

Module 3: Your database is a contract



+



= "the data is structurally correct"

The Spreadsheet Problem

employee_name	dept_name	dept_location	project_name	project_budget	hours_worked
Omar Haddad	Engineering	Building A	Atlas	50000	120
Layla Mansour	Engineering	Building A	Atlas	50000	80
Ahmad Saleh	Engineering	Building A	Atlas	50000	95

Problem: Data Corruption

The Update Anomaly

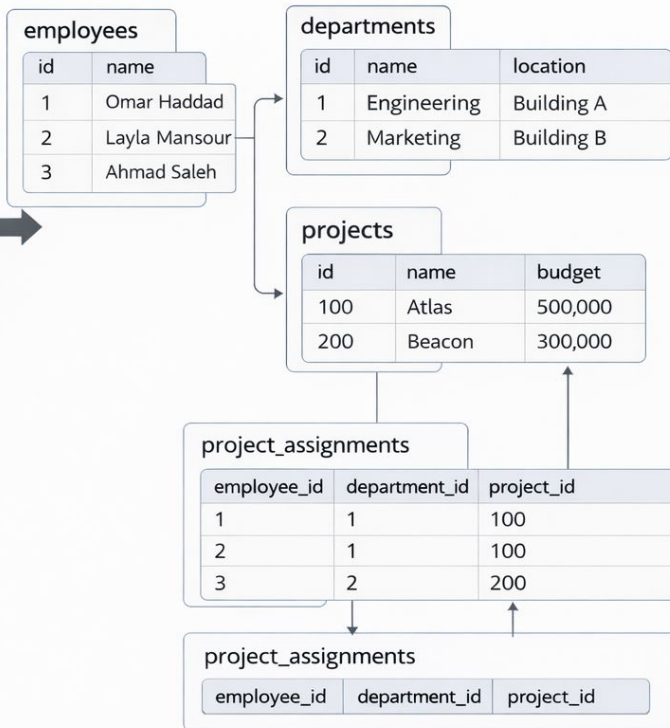
employee_name	dept_name	dept_location	project_name	project_budget	hours_worked
Omar Haddad	Engineering	Building A	Atlas	50000	120
Layla Mansour	Engineering	Building A	Atlas	50000	80
Ahmad Saleh	Engineering	Building A	Atlas	30000	95
Noor Kassab	Marketing	Building B	Beacon	30000	95
Khaled Issa	Marketing	Building B	Beacon	30000	60
Dina Farah	Engineering	Building A	Atlas	50000	110

Solution: Decomposition

Before

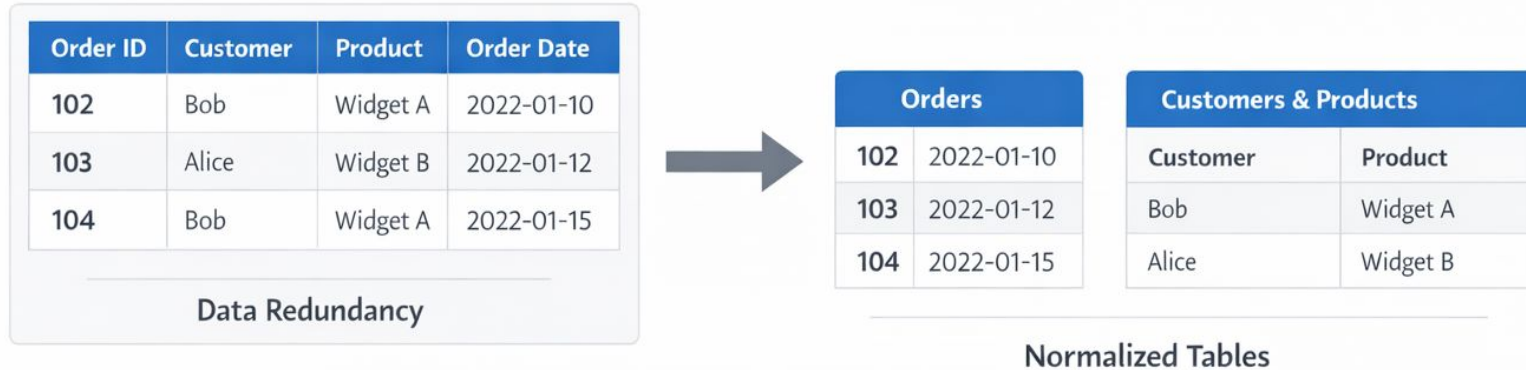
employee_name	dept_name	dept_location	project_name	hours_worked
Omar Haddad	Engineering	Building A	Atlas	500,000
Layla Mansour	Engineering	Building A	Atlas	500,000
Ahmad Saleh	Engineering	Building A	Atlas	500,000
Noor Kassab	Marketing	Building B	Beacon	300,000
Khaled Issa	Marketing	Building B	Beacon	300,000
Dina Farah	Engineering	Building A	Atlas	500,000

After



- One spreadsheet → four tables
- Each fact stored once
- Relationships connect them

Decomposition is part of Normalization



Normalization

A step-by-step process for eliminating redundancy



First Normal Form (1NF)

Before

Name	Skills
Omar	Python, SQL, Docker
Layla	Java, HTML
Ahmad	Excel, Power BI



After

Name	Skill
Omar	Python
Omar	SQL
Omar	Docker
Layla	Java
Layla	HTML
Ahmad	Excel
Ahmad	Power BI

Second Normal Form (2NF)

Before

employees (PK: employee_id)		
employee_id	employee_name	dept_name (FK)
1	Omar	Engineering
2	Layla	Engineering
3	Ahmad	Finance
4	Noor	Finance



After

employee_id	employee_name
1	Omar
2	Layla
3	Ahmad
4	Noor

employee_name	dept_name
Omar	Engineering
Layla	Engineering
Ahmad	Finance
Noor	Finance

Third Normal Form (3NF)

Before



transitive dependency

After

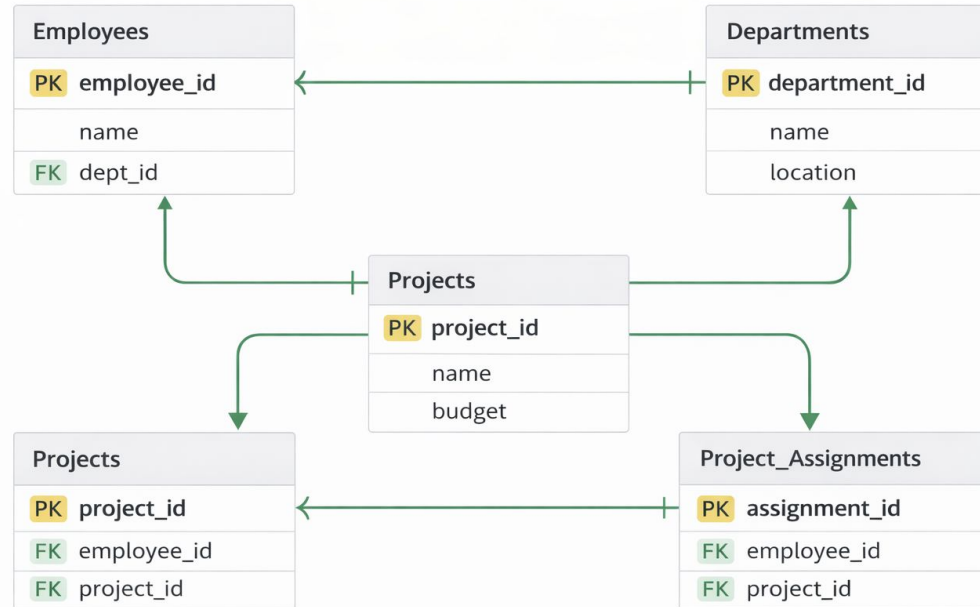
Employees

employee_id	dept_name
101	Engineering

Departments

dept_name	dept_location
Engineering	Building C

The Normalized Schema



What prevents someone from assigning an employee to a department that does not exist?

Primary Keys

○	id	name	dept_name
1	1	Noor	Engineering
2	2	Khaled	Marketing
3	3	Layla	HR
4	4	Tariq	Finance

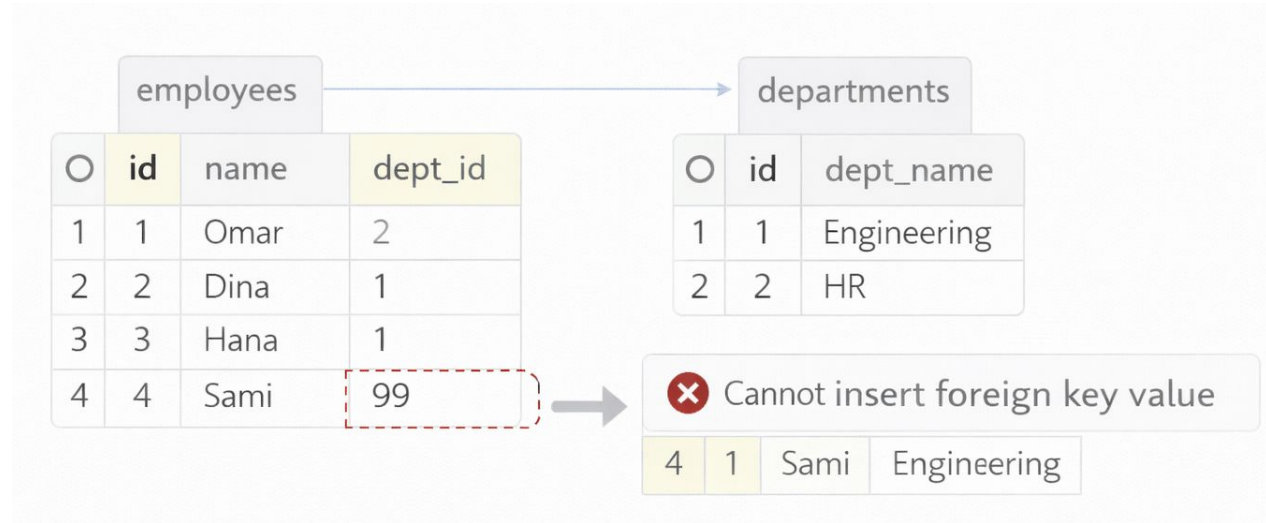


✗ Cannot insert duplicate key value			
○	id	name	dept_name
1	1	Sami	Engineering

✗ d 1 Sami Engineering

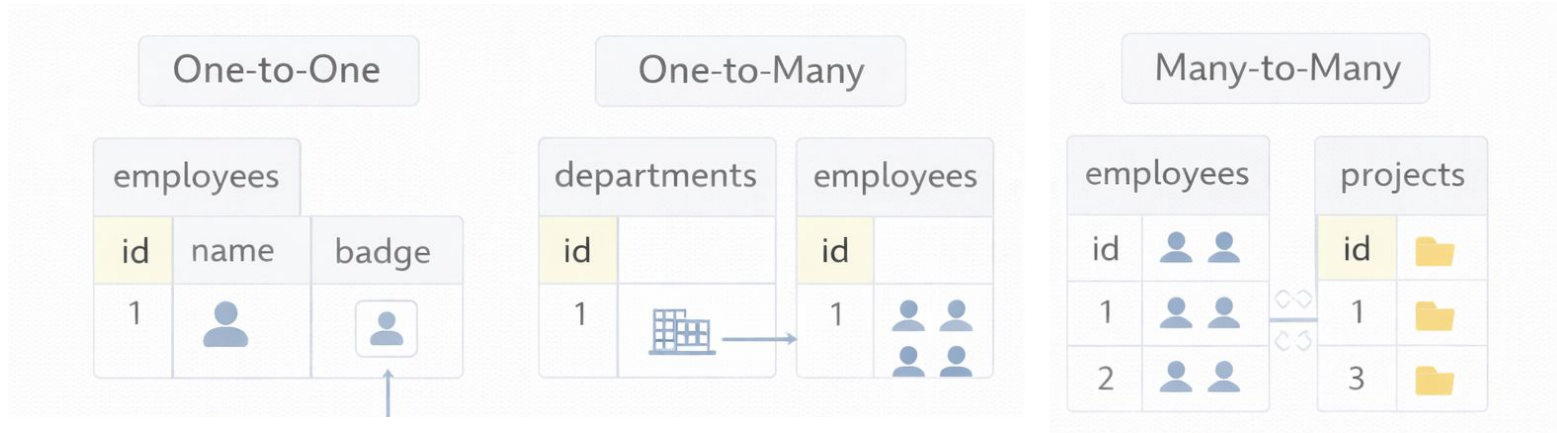
Foreign Keys

A foreign key enforces valid relationships between tables and prevent broken or inconsistent data.

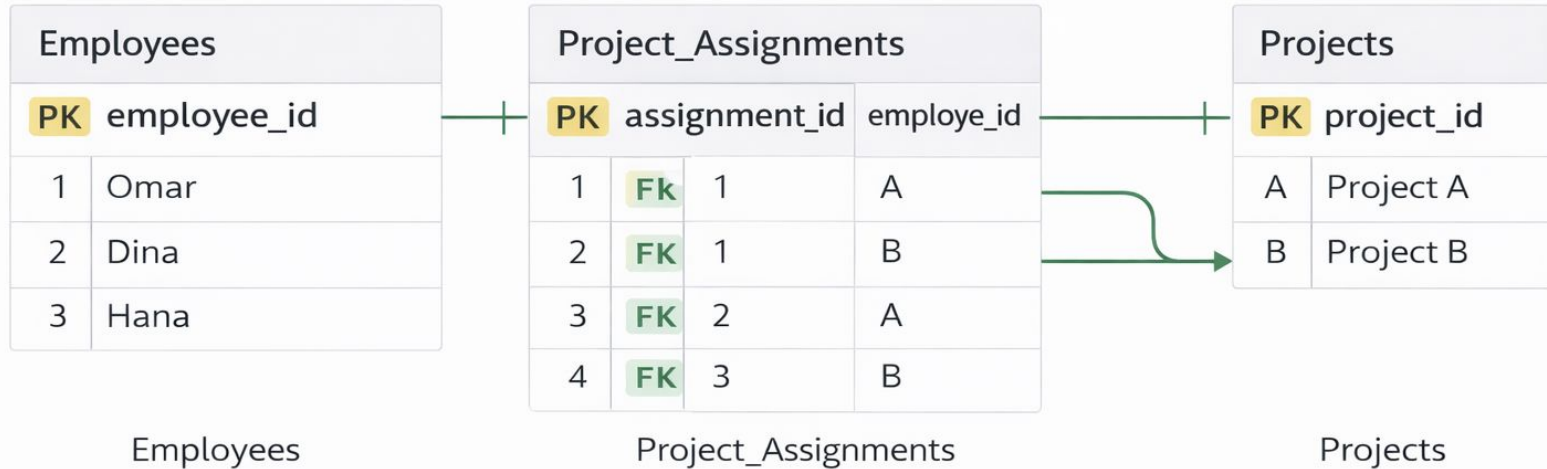


Relationship Types

Tables connect through three relationship patterns



Bridge Tables



The SQL Mindset

SQL is not commands to memorize — it is a way to ask questions about relationships

Python (How)

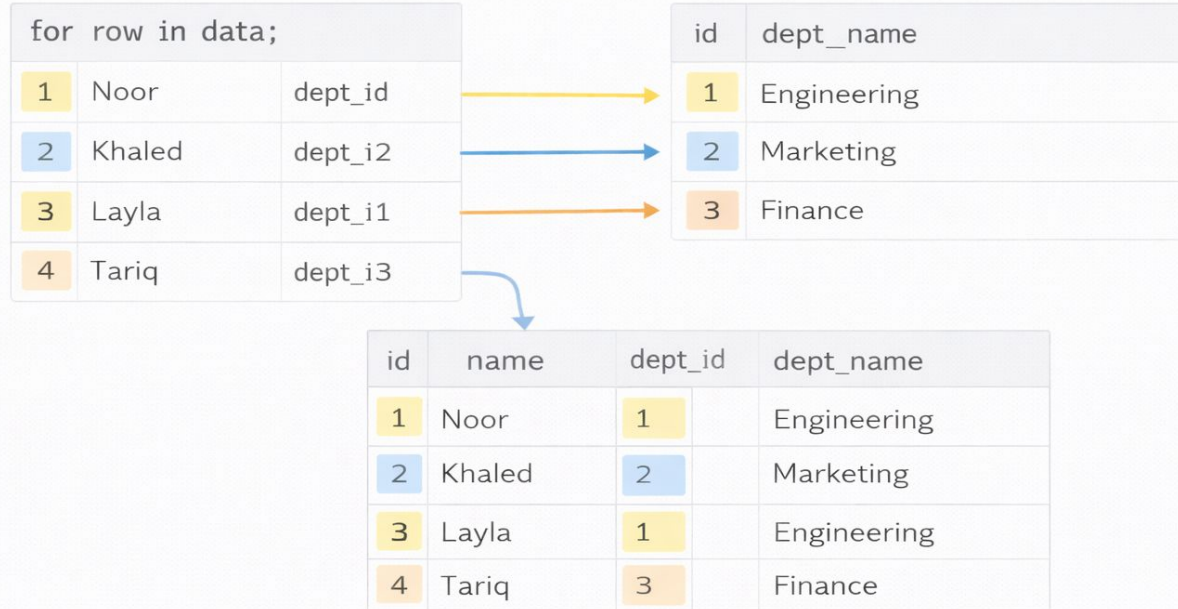
```
for row in employees:  
    if row['department'] == 'Engineering':  
        total_salary += row['salary']
```

SQL (What)

```
SELECT department,  
        SUM(salary)  
FROM employees  
WHERE department = 'Engineering';
```

Declarative programming

JOIN



LEFT JOIN

name	proj_id	proj_name	proj_deadline
Omar	10	Alpha	May 15
Layla	20	Beta	Jun 1
Ahmad	NULL	NULL	NULL
Noor	NULL	NULL	NULL
Khaled	NULL	NULL	NULL
Hana	NULL	NULL	NULL

Unassigned employees shown as NULL

GROUP BY

Collapses many rows into one row per group and applies a calculation

‘How much does each department spend on salaries?’



```
SELECT d.name, SUM(e.salary)
FROM employees e
JOIN departments d ON e.dept_id = d.id
GROUP BY d.name
```

Query Execution Order

SQL processes your query in a specific order — not the order you write it



Window Functions

GROUP BY

department	max_salary
Engineering	10000
Marketing	9000
Finance	8000



Window Function

department	name	salary	rank
Engineering	Noor	10000	1
Marketing	Knaled	9000	1
Finance	Tariq	8000	1
Finance	Fatima	4000	2
	Jihan	3000	1
		3000	3

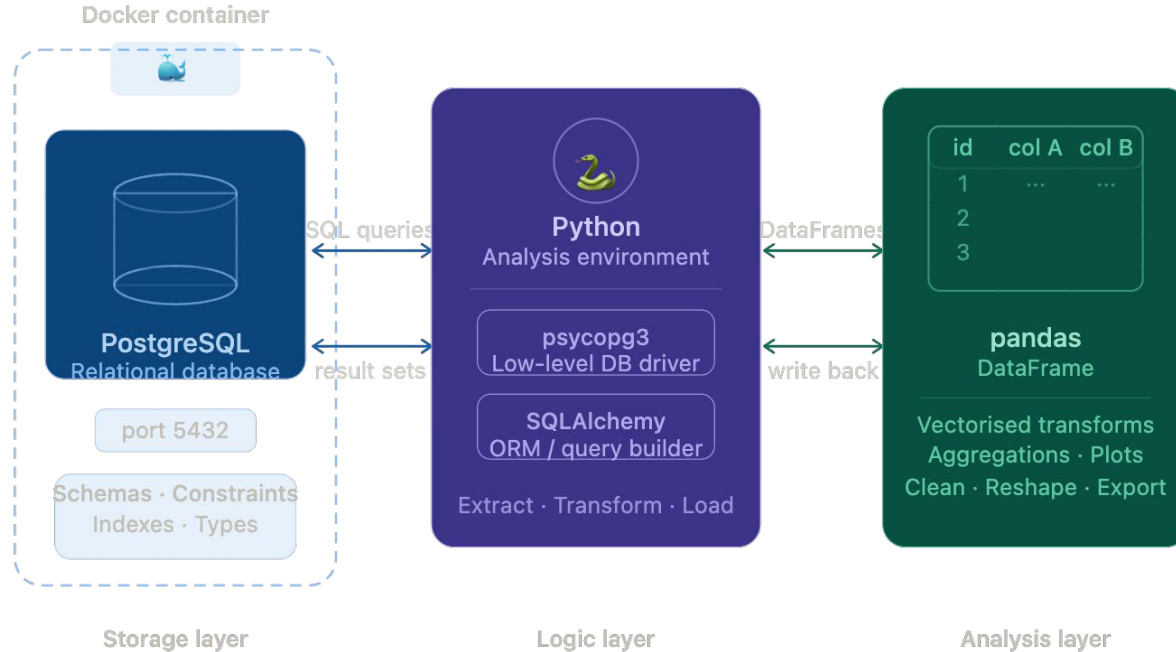
Original rows preserved → rank

ROW_NUMBER Example

PARTITION BY department ORDER BY salary DESC

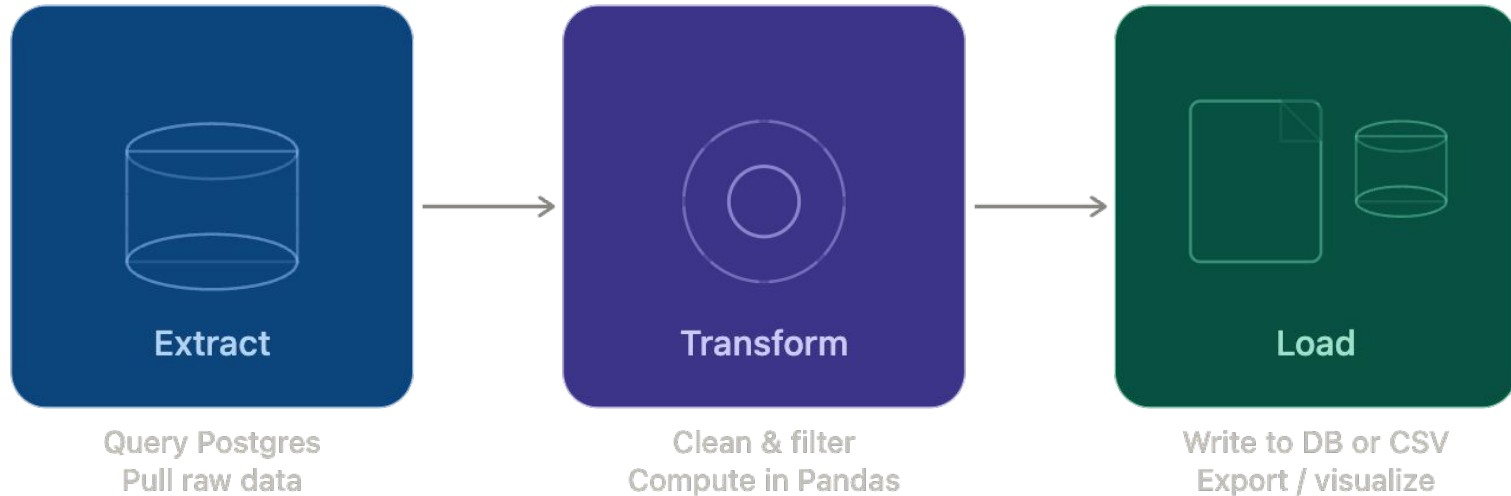
	name	department	salary	row number
{	Layla	Engineering	4500	1
	Omar	Engineering	4000	2
{	Noor	Marketing	3600	1
	Ahmad	Marketing	3500	2
{	Khaled	Sales	3000	1
	Hana	Sales	3000	2
	Khaled	Sales	3000	1
	Hana	Sales	3000	2

Python-Postgres Architecture



Python ↔ PostgreSQL integration architecture

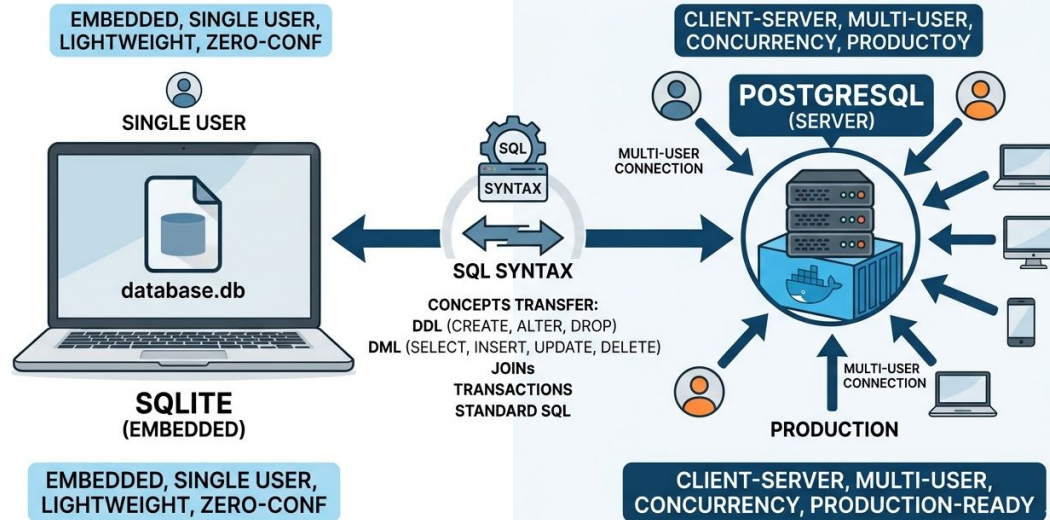
The ETL Pattern



Extract → Transform → Load pipeline

SQLite vs Postgres

SQLITE VS. POSTGRESQL: APPLICATION SCOPE & USER MODEL



Why does normalization matter?

What do keys guarantee?

What does NULL mean in a LEFT JOIN?

Big Picture



Messy Spreadsheet

Engineering is listed in 50 different rows

employee_name	dept_name	dept_location	project
Omar Haddad	Engineering	Building A	Migration
Layla Mansour	Engineering	Building A	API Redesign
! Row 3	Engineering	Building A	Data Pipeline
! Row 4	Engineering	Building A	Analytics
! Row 5	Engineering	Building A	Backend
! ...	...	...	...
! Row 48	Engineering	Building B	DevOps
! Row 49	Engineering	Building B	Platform
! Row 50	Engineering	Building A	Security



50 updates

Hope you do not miss one



Database

Engineering moves in one row — not fifty

departments		
dept_id	name	location
1	Engineering	Building C
2	Marketing	Building B
3	Sales	Building C

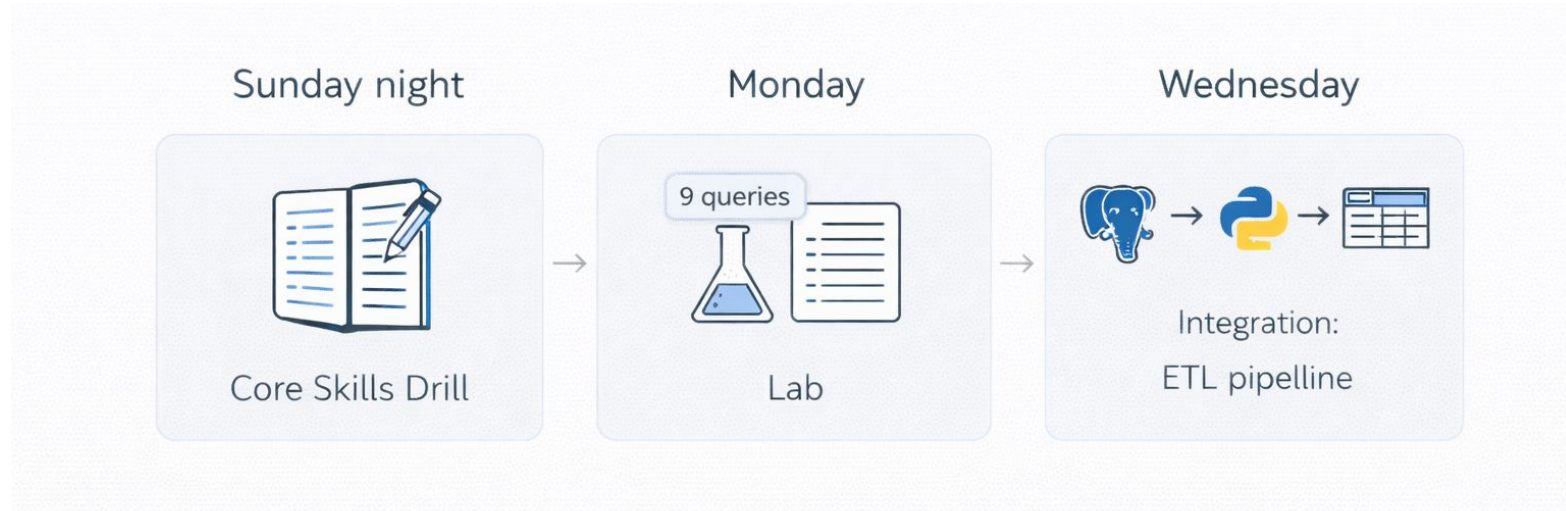


1 update

Guaranteed correct

You are no longer working with files.

What Happens Next



Questions?